Error Recovery in Parsers

Panic Mode vs. Minimal Insertion

Compiler Design · Recursive Descent Parsing · Based on "Don’t Panic! Better, Fewer, Syntax Errors for LR Parsers" Diekmann & Tratt (2020)

Made with GAMMA

What Is Parsing — and Why Does It Fail?

Parsing

Transforms a stream of tokens into a structured parse tree that the compiler can analyze.

Syntax Errors

Occur when the token stream violates the language grammar — e.g., missing operators or unmatched parentheses.

The Real Challenge

A parser must not just detect the first error — it must **recover** and continue to report further errors meaningfully.

Parsing is the second phase of compilation. Errors here are structural, not just misspellings — the token sequence itself breaks the grammar rules.

Made with GAMMA

The Problem: Cascading Errors

One real error can produce dozens of spurious messages. Poor recovery forces programmers to fix errors blindly — the root cause is buried under noise. The goal of any recovery strategy is to **minimize cascading errors** while staying in sync with the grammar.

Made with GAMMA

Project Overview

Expression Evaluator

Parses and evaluates arithmetic expressions: `3 + 4 * 2`

Dual-Mode Parser

A single recursive descent parser switches between **Panic Mode** and **Minimal Insertion** at runtime.

Pipeline

Source string → **Tokenizer** → Token list → **Parser** → AST / result

Made with GAMMA

Tokenizer & Parser Design

Grammar

```
E → T (('+' | '-') T)*
T → F (('*' | '/') F)*
F → NUMBER | '(' E ')'
```

Token Types

- NUMBER
- + / −
- * / /
- EOF

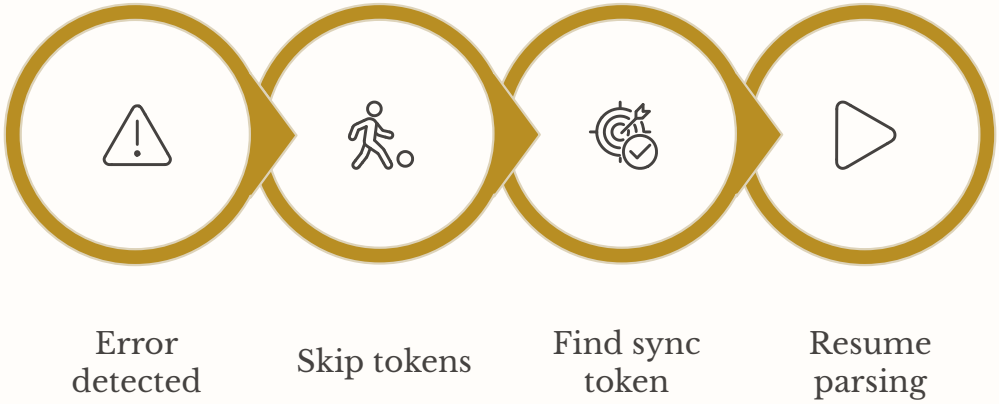
Recursive Descent

Each grammar rule maps to a Python function: `parse_E()`, `parse_T()`, `parse_F()`.

The helper `eat(token_type)` consumes an expected token or triggers recovery.

Made with GAMMA

Panic Mode Recovery



How It Works

- On error, **discard tokens** until a known synchronization point is found
- Sync tokens: `+`, `-`, `EOF`
- Parser re-enters the grammar at a safe rule

✅ Pros

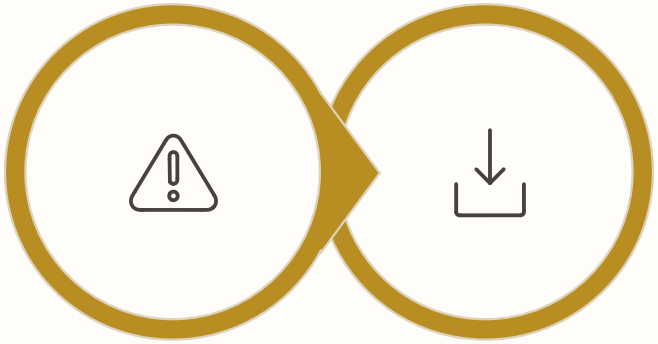
Simple to implement · Fast recovery · Predictable behavior

❌ Cons

Discards valid tokens · Loses context · May skip too much

Made with GAMMA

Minimal Insertion Recovery



Error Detected

Inject Token

How It Works

- On error, **insert a synthetic token** (e.g., `NUMBER(0)`) to satisfy the grammar
- Parsing continues *without* discarding any real tokens
- Error is logged; the fabricated value propagates

✅ Pros

Preserves context · Better continuity · Fewer lost tokens

❌ Cons

Assumes a default value · May propagate incorrect results

Made with GAMMA

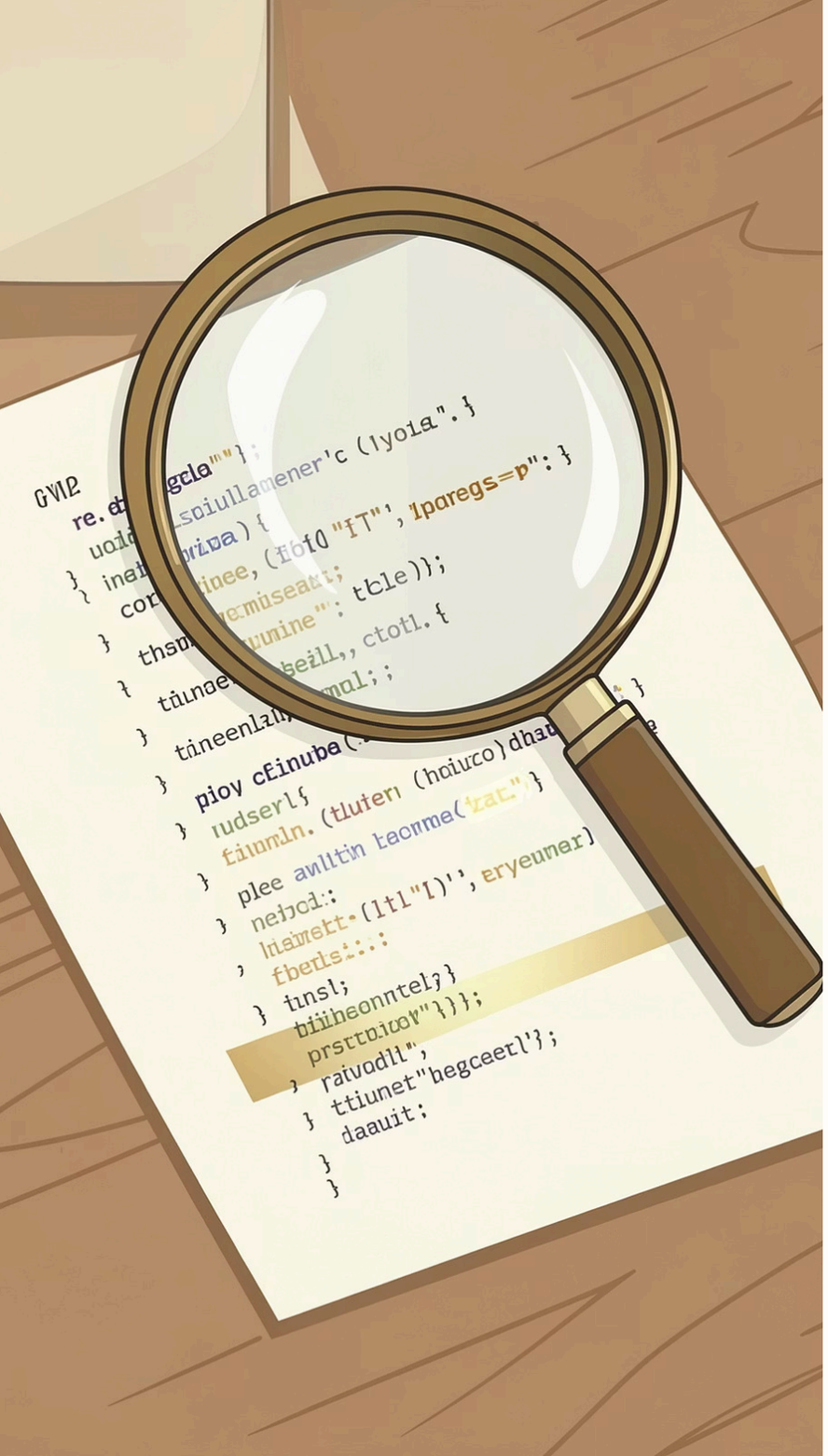
Comparison & Paper Insight

Criterion	Panic Mode	Minimal Insertion
Strategy	Skip tokens	Insert tokens
Cascading errors	Higher risk	Lower risk
Token preservation	Low	High
Implementation	Simple	Moderate
Error message quality	Moderate	Better
Result correctness	Safe (skips)	May assume

Diekmann & Tratt (2020) — "Don't Panic! Better, Fewer, Syntax Errors for LR Parsers" LR parsers should produce **fewer, higher-quality** errors rather than flooding output. Prioritize error locality and minimize spurious cascades.

Our dual-mode implementation directly reflects this: minimal insertion yields fewer follow-on errors, while panic mode offers a simpler safety net.

Made with GAMMA



Example & Conclusion

Faulty Input

```
3 + * 5
```

Panic Mode Output

```
Error at '*': unexpected
token
Skipping to next sync point...
Result: 8
```

Minimal Insertion Output

```
Error at '*': missing operand
Inserted: NUMBER(0)
Result: 3 + 0 * 5 = 3
```

Final Takeaway

→ Use Panic Mode

When simplicity matters and losing a sub-expression is acceptable.

→ Use Minimal Insertion

When continuity and reduced cascading errors are the priority.

→ Paper Verdict

Fewer, better-targeted errors always outperform a flood of cascading messages.

Made with GAMMA